

Complete Beginner Guide: JWT + AI Microservices Project

Author: Cursor assistant

Audience: Student with little/no programming background

Goal: Help you present this project confidently in 2 days

1) Big Picture (What this project is)

This project is a **security-focused login system** split into small parts called **microservices**.

In plain words:

- A user tries to log in.
- The system checks if the password is correct.
- The system also asks an AI model: "Does this login look suspicious?"
- If suspicious, the account is blocked for 5 minutes.
- If safe, user gets a JWT token and can access protected APIs.
- Admin can watch all this in a live dashboard.

So this is not only "login"; it is "login + security intelligence + monitoring".

2) Why people need something like this

Real systems face attacks:

- brute-force (many wrong passwords quickly)
- credential stuffing (trying leaked passwords)
- impossible travel (same user from very far locations in short time)
- bot traffic

This project demonstrates how to reduce risk using:

- **bcrypt** for secure password storage
- **JWT** for stateless authentication across services
- **AI anomaly detection** for suspicious behavior
- **rate limiting** to slow abusive traffic
- **live dashboard** for admin visibility

3) Core technologies in simple language

JWT

A JWT is like a signed ID card for APIs:

- Auth service creates token after login
- Resource services verify token using shared secret
- If token missing/invalid/expired -> 401 Unauthorized

bcrypt

Password is not stored as plain text.
It is stored as a strong one-way hash.

Isolation Forest (AI)

AI learns what "normal logins" look like and gives a risk score for new logins.

SignalR

Used so dashboard gets real-time events without refreshing constantly.

Docker

Only PostgreSQL database runs in a Docker container.

4) Architecture (Who does what)

Your system has these runtime parts:

1. `auth-service` (ASP.NET Core, port 5038)
2. `resource-service-1` (ASP.NET Core, port 5226)
3. `resource-service-2` (ASP.NET Core, port 5295)
4. `ai-service` (FastAPI Python, port 8001)
5. `dashboard` (Blazor Server UI, port 5080)
6. `postgres-db` (Docker, port 5432)

Optional HTTPS ports with self-signed cert:

- auth: 5001
- resource-1: 5003
- resource-2: 5004

5) End-to-end request flow

Login flow

1. Client sends `POST /api/auth/login` to auth-service.
2. Auth checks rate limit.
3. Auth verifies password with bcrypt hash.
4. Auth builds behavioral features:
 - failed attempts
 - IP change
 - country change
 - impossible travel
 - user-agent change
5. Auth sends features to AI `/score`.
6. AI returns anomaly/riskScore.
7. If suspicious:
 - account blocked for 5 minutes
 - event sent to dashboard via SignalR
 - login denied
8. If safe:
 - JWT generated (30 min expiry)
 - event logged and shown in dashboard
 - token returned to client

API access flow

1. Client calls resource `/secure` endpoint with JWT.
2. Resource validates JWT (issuer/audience/key/expiry).
3. If valid -> 200 OK.
4. If invalid/missing -> 401 Unauthorized.

6) File/folder structure and why each exists

High-level structure:

```
```text
mikro_sherbimet/
├── auth-service/
│ ├── resource-service-1/
│ └── resource-service-2/
├── ai-service/
├── dashboard/
├── scripts/
├── models/
├── docs/
├── certs/
├── docker-compose.yml
└── README.md
```
```

`auth-service/` (main business/security logic)

Important files:

- `Program.cs`
 - wires database, services, CORS, Swagger, SignalR, ports
- `Controllers/AuthController.cs`
 - register/login, JWT generation, AI call, block logic
- `Controllers/MonitorController.cs`
 - dashboard APIs (`events`, `blocked`, `unblock`)
- `Services/GeoLocationService.cs`
 - IP -> geo lookup + distance calculation
- `Services/LoginRateLimiter.cs`
 - token bucket rate limiting per key (email+IP)
- `Services/SuspiciousLoginDetector.cs`
 - fallback rule-based anomaly detection
- `Data/ApplicationDbContext.cs`
 - database context (Users, LoginEvents)
- `Models/User.cs`
 - user entity with security tracking fields
- `Models/LoginEvent.cs`
 - each login event + AI metadata
- `Migrations/\*`
 - DB schema history
- `appsettings.json`
 - JWT settings, DB connection, AI URL

`resource-service-1/` and `resource-service-2/`

Both do same idea:

- validate JWT
- expose `/public` and `/secure`
- reject unauthorized calls

Main file in each: `Program.cs`.

`ai-service/`

- `main.py`
 - FastAPI app with `/health` and `/score`
 - loads trained model and threshold from `models/`
- `train.py`
 - generates synthetic training data
 - trains Isolation Forest
 - saves model + threshold
- `requirements.txt`

- Python dependencies

`dashboard/` (UI for demonstration)

Main files:

- `Program.cs`
 - configures Blazor and HttpClient
- `Shared/MainLayout.razor`
 - sidebar/navigation layout
- `Pages/Home.razor`
 - project overview
- `Pages/Login.razor`
 - register/login UI + stores JWT in session state
- `Pages/ApiTest.razor`
 - test resource APIs with/without token
- `Pages/Index.razor` (route `/monitor`)
 - real-time events, filters, blocked accounts
- `Services/SessionState.cs`
 - temporary in-memory token/email for UI session
- `wwwroot/css/app.css`
 - custom styling
- `Pages/\_Host.cshtml`
 - HTML shell and Bootstrap include

`scripts/` (automation and demos)

- `run-all-services.sh`
 - starts DB and all services in background
 - runs integration tests
- `run-integration-tests.py`
 - checks all key requirements automatically
- `test-jwt.sh`
 - demonstrates token behavior quickly
- `brute\_force\_simulator.py`
 - simulates rapid wrong password attacks
- `impossible\_travel\_simulator.py`
 - simulates far-geo logins
- `normal\_login\_tuning.py`
 - helps threshold tuning and false-positive check
- `generate-certs.sh`
 - creates self-signed certs for TLS demo

`models/`

- `isolation\_forest.pkl` (trained model)
- `model\_config.json` (threshold)

`docs/`

- `DEMO\_CHECKLIST.md` (presentation sequence)
- `PERGJIGJET\_SHKRIMORE.md` (theory answers)
- this guide file

`docker-compose.yml`

Defines PostgreSQL container (`postgres:16`) used by auth-service.

7) Code walkthrough by service

Auth Service deep explanation

`Program.cs` responsibilities

- Connects to PostgreSQL via EF Core.
- Registers:
 - `GeoLocationService`
 - `LoginRateLimiter`
 - named HTTP client `"ai"` for AI calls.
- Enables SignalR hub `/monitorHub`.
- Exposes HTTP on `5038`; HTTPS `5001` if cert exists.

`AuthController.cs` responsibilities

Register endpoint

- Checks if email exists.
- Hashes password using bcrypt.
- Saves user.

Login endpoint

Main security pipeline:

1. gets client IP (`X-Forwarded-For` supported)
2. rate-limit check
3. user lookup
4. check temporary block (`BlockedUntil`)
5. bcrypt verify password
6. compute behavior features
7. call AI `/score`
8. merge AI + deterministic rules (`SuspiciousLoginDetector`)
9. if anomaly -> block 5 minutes + reject
10. if safe -> generate JWT and return token
11. always save login event and push SignalR event

Health endpoint

`GET /api/auth/health` for quick service checks.

`MonitorController.cs`

Provides dashboard data:

- `GET /api/monitor/events` with filters
- `GET /api/monitor/blocked`
- `POST /api/monitor/unblock?email=...`

`SuspiciousLoginDetector.cs`

Reason this file exists:

- If AI service is down or model score is borderline, demo should still detect obvious attacks.
- Rule checks:
 - impossible travel flag
 - failed attempts threshold (≥ 8)
 - low success rate with many failures

Resource Services deep explanation

Each resource service:

- validates JWT using same secret (`Jwt:Key`)

- has:
 - `/public` (no auth)
 - `/secure` (auth required)
- returns claims info from JWT to prove identity propagation.

These services prove distributed auth works across microservices.

AI Service deep explanation

`train.py`

Creates training dataset with:

- normal logins
- normal variations
- brute-force anomalies
- impossible-travel anomalies

Trains Isolation Forest and saves model.

`main.py`

At startup:

- loads model and threshold.

Endpoints:

- `GET /health`
- `POST /score`:
 - receives features
 - computes decision function score
 - flags anomaly if `score < threshold`.

Dashboard deep explanation

Purpose:

- show what happens in system live (for admin + professor demo).

Pages:

- Home: overview
- Login: register/login without Postman
- API Test: verify 401/200 behavior
- Monitor: live login stream + risk score + blocked users

SignalR:

- Monitor page opens connection to auth hub `/monitorHub`.
- New login events appear immediately.

8) Database model in simple terms

`Users` table (from `User.cs`)

Stores identity + security state:

- Email, Username
- PasswordHash (bcrypt)
- FailedLoginAttempts
- BlockedUntil
- LastLogin geo/IP/time fields
- LoginCount

`LoginEvents` table (from `LoginEvent.cs`)

Stores audit log for each attempt:

- success/fail
- ip, country, city, coordinates
- isAnomaly, riskScore
- timestamp

9) Security design choices and reasons

1. **bcrypt instead of SHA-256**
 - slower by design -> harder brute-force offline.
2. **JWT stateless auth**
 - easy to validate in each resource service.
3. **Short token expiry (30 min)**
 - limits damage window.
4. **Rate limiting**
 - reduces rapid abuse.
5. **AI + rules**
 - AI catches behavior patterns, rules guarantee obvious attack detection.
6. **Temporary blocking**
 - quick response without permanent lock.
7. **TLS support**
 - secures token in transport.

10) How to run everything (step-by-step)

From project root:

```
```bash
cd /home/dion/mikro_sherbimet
```
```

First-time setup:

```
```bash
docker compose up -d
dotnet ef database update --project auth-service/auth-service.csproj
python3 -m venv ai-service/.venv
ai-service/.venv/bin/pip install -r ai-service/requirements.txt
ai-service/.venv/bin/python ai-service/train.py
dotnet build JwtAiSystem.sln
```
```

Start all services:

```
```bash
./scripts/run-all-services.sh
```
```

Open UI:

```
- Dashboard: http://localhost:5080
- Swagger: http://localhost:5038/swagger
```

Run full health+integration checks:

```
```bash
python3 scripts/run-integration-tests.py
```
```

Expected: `13 passed, 0 failed`

11) How to explain this project in presentation (easy script)

You can say this:

1. "My project is a microservice login security system."
2. "Auth service handles users, passwords, and JWT."
3. "AI service scores each login for suspicious behavior."
4. "If risk is high, account is blocked for 5 minutes."
5. "Two resource services validate JWT independently."
6. "Dashboard shows events and blocked users in real time."
7. "I tested everything with automated integration tests."

Then demo:

1. normal register/login -> token
2. API test no token -> 401
3. API test with token -> 200
4. brute-force script -> blocked
5. monitor page updates live

12) Common issues and fixes

Ports already used

```
```bash
fuser -k 8001/tcp 5038/tcp 5226/tcp 5295/tcp 5080/tcp
```
```

AI model missing

```
```bash
ai-service/.venv/bin/python ai-service/train.py
```
```

DB not reachable

```
```bash
docker compose up -d
docker compose ps
```
```

Services stopped

Run:

```
```bash
./scripts/run-all-services.sh
```
```

13) What is still "demo-level" vs production-level

Great for university:

- architecture, security concepts, testing, dashboard, AI demo.

Would need more for production:

- secrets in vault (not appsettings)
- stronger audit logging
- distributed rate-limiter (Redis)
- robust ML pipeline with real data and retraining
- better user lifecycle (password reset, MFA)

14) Quick glossary (for non-programmer)

- ****API****: a URL endpoint another program calls.
- ****Microservice****: small independent backend app focused on one job.
- ****JWT****: signed token proving user identity.
- ****Hash****: irreversible transformed value (for passwords).
- ****Endpoint****: specific API path like `/api/auth/login`.
- ****Anomaly****: behavior far from normal pattern.
- ****SignalR****: live server-to-browser updates.
- ****Docker container****: isolated packaged runtime for services.

15) Final one-paragraph summary

This project is a complete mini security platform: it authenticates users securely, detects suspicious login behavior with AI and fallback security rules, protects APIs with JWT across microservices, and gives administrators a live monitoring dashboard. It demonstrates practical distributed-systems security concepts (JWT sharing, stateless auth, rate limiting, geolocation checks, anomaly scoring, temporary account blocking, real-time monitoring) in a way that is easy to demo and evaluate academically.